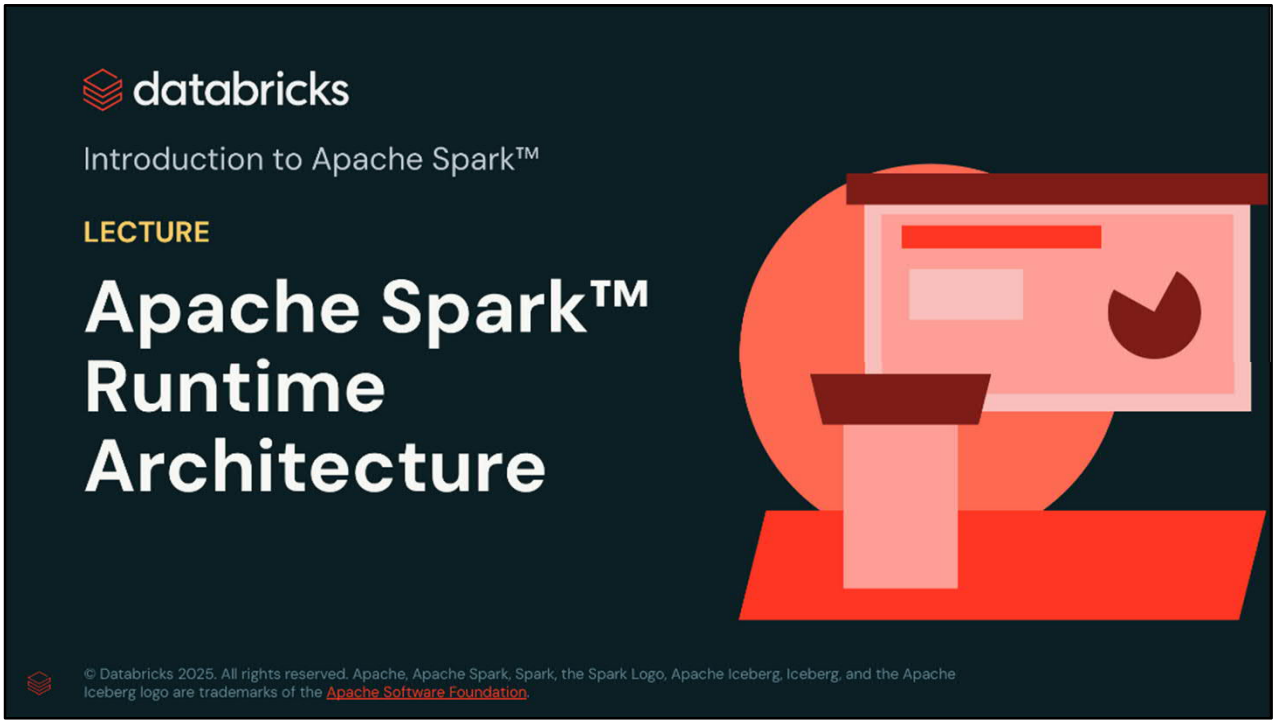
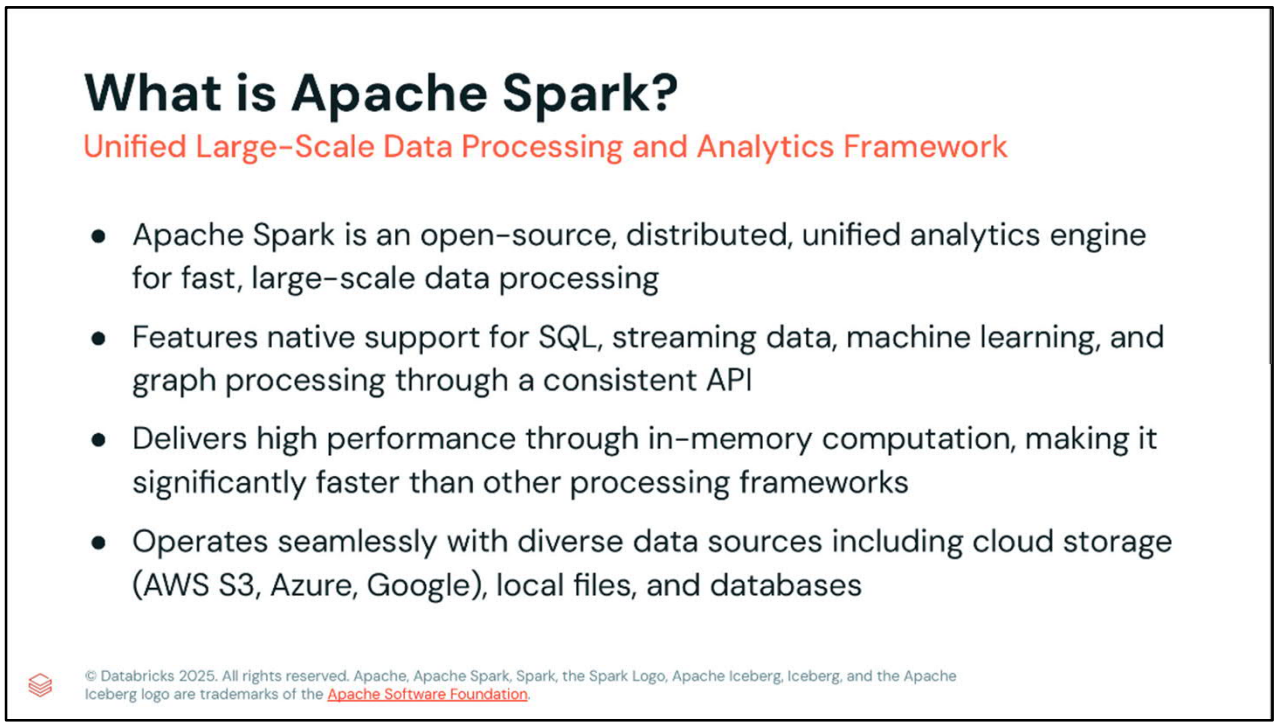


This lecture provides an in-depth overview of the Apache Spark™ Runtime Architecture, its core components, execution flow, and how Spark operates within Databricks clusters.

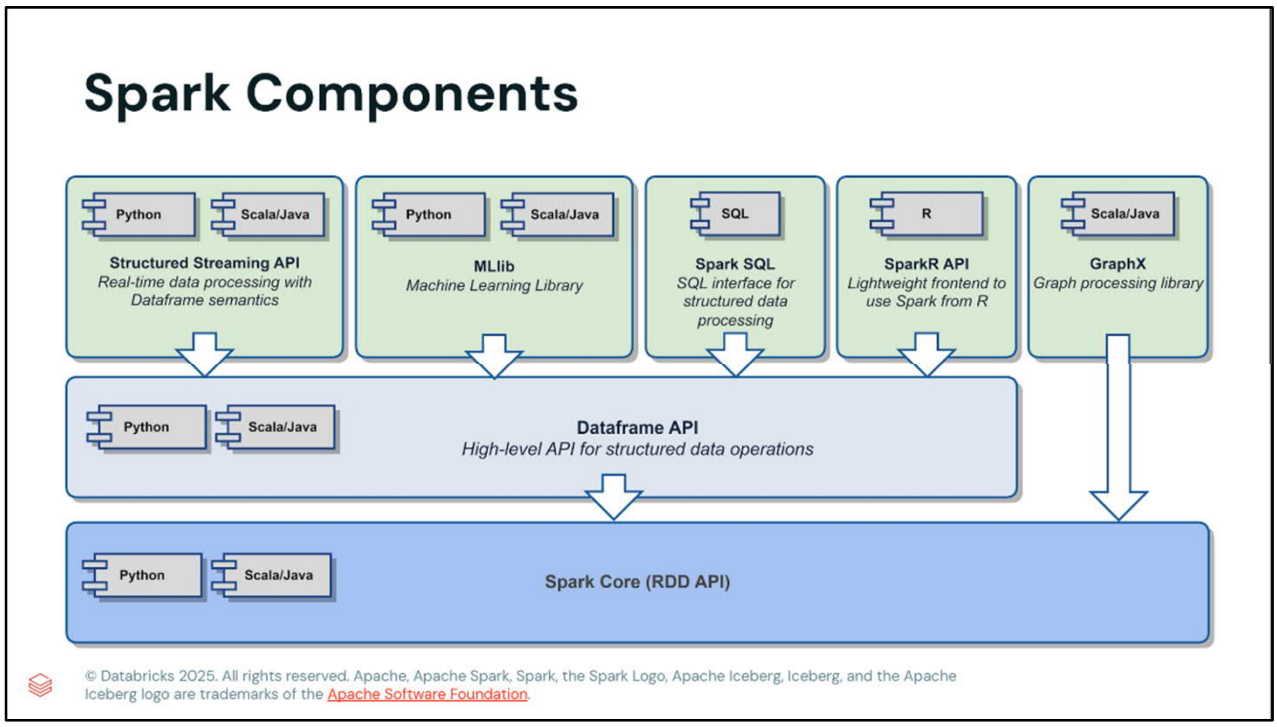
Lesson Learning Objectives

- Identify the roles of the Driver, Workers, and Executors in Spark applications.
- Understand the role of the Spark Driver and learn how SparkSession serves as the unified entry point for Spark applications.
- Understand how Worker nodes execute tasks and how resources are allocated within Spark applications.
- Utilize the Spark UI to monitor application progress, DAG visualizations, and resource allocation.

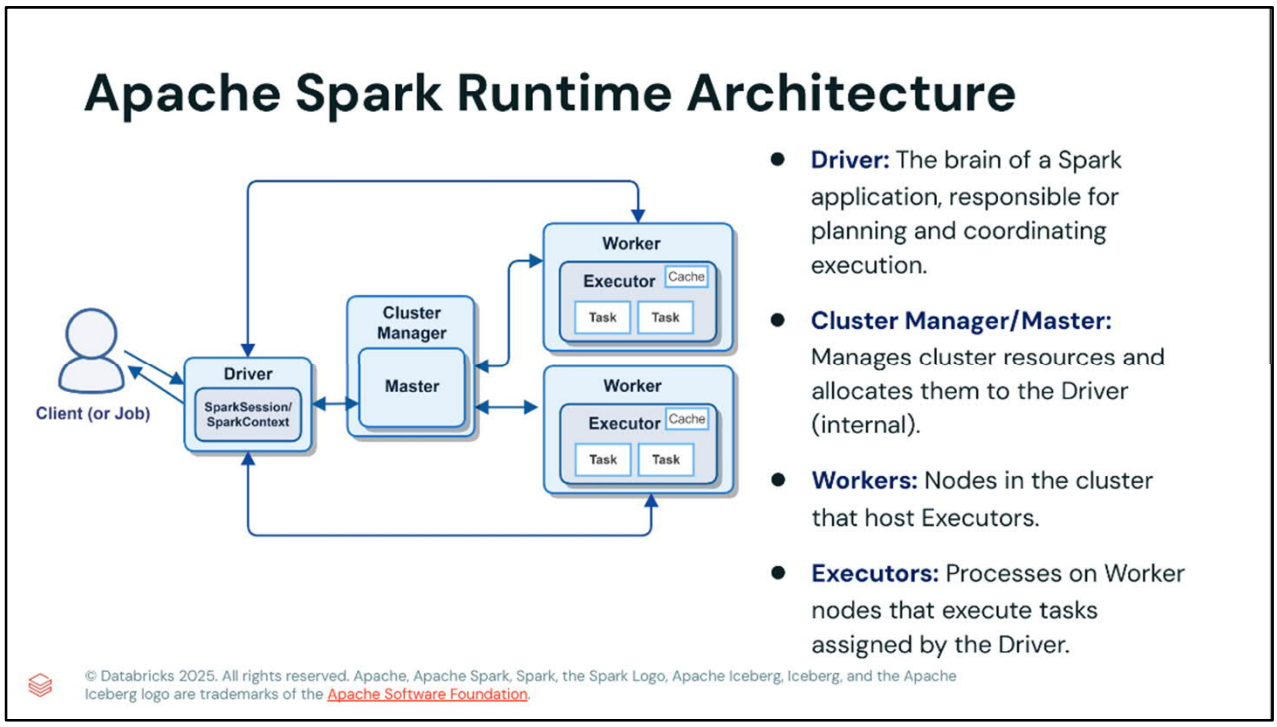
By the end of this lecture, you will be able to describe the runtime architecture and different roles within a Spark application, including the driver, workers, and executors; explain the Spark driver and the Spark session as the entry point; understand executors and resource management; describe how tasks and work are distributed; and visualize a Spark application at runtime using the Spark UI.



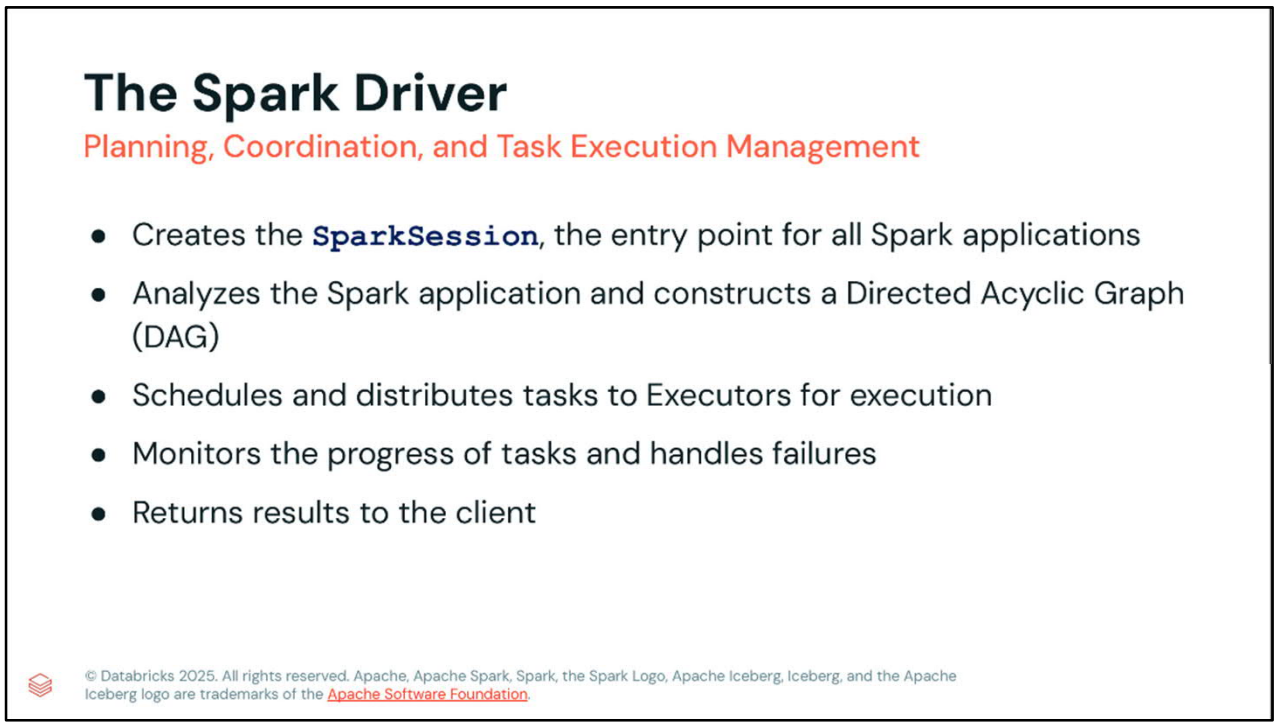
Apache Spark is a unified analytics framework providing a consistent interface for handling big data across multiple domains. It is open-source and distributed, built for large-scale processing and horizontal scalability. It has native support for multiple interfaces, including SQL, streaming, machine learning, graph processing, and more. It is designed for high performance, built around in-memory processing, which is significantly faster than disk-based systems that preceded it. It seamlessly integrates into cloud environments, enabling reading and writing to and from cloud storage.



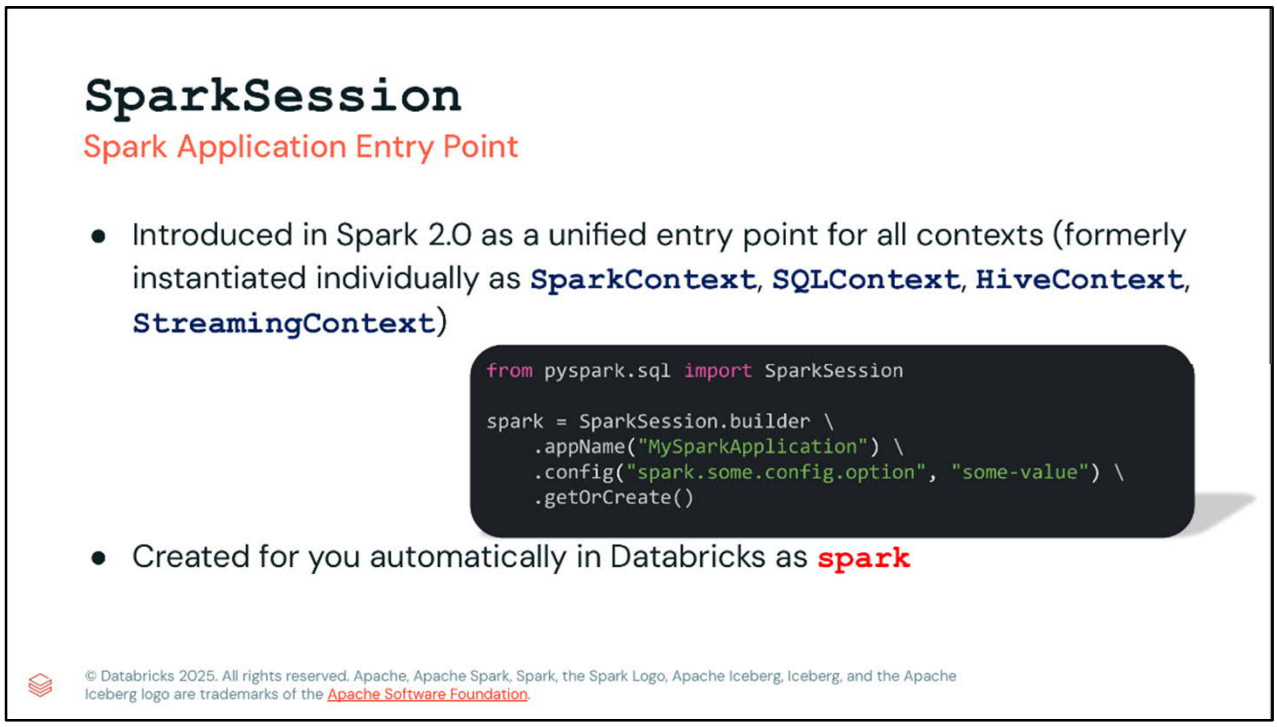
The Spark Core Engine provides the foundation for all Spark applications, handling memory management, fault recovery, scheduling, and task distribution. It is exposed through high-level APIs such as the DataFrame API, RDDAPI, and SQL API, which are available across multiple languages including Python, Scala, Java, and R. Specialized libraries built on top of the core engine, such as Spark SQL, MLlib, GraphX, and Structured Streaming, provide advanced functionalities for various use cases.



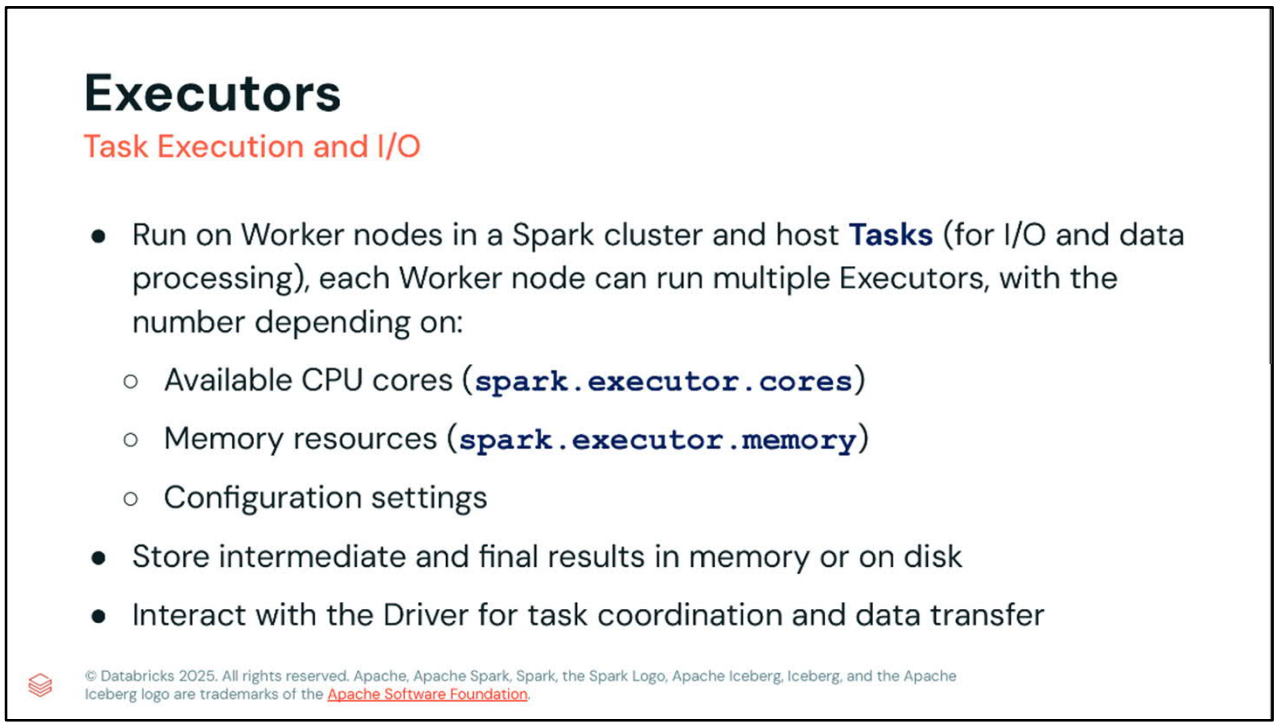
The Driver is the central component that manages the application lifecycle and execution. The Cluster Manager (Master) allocates resources and assigns tasks to workers; on Spark/Databricks clusters. Workers perform the tasks assigned by the driver, execute code, and return results. Executors run on worker nodes, handling task execution and caching data.



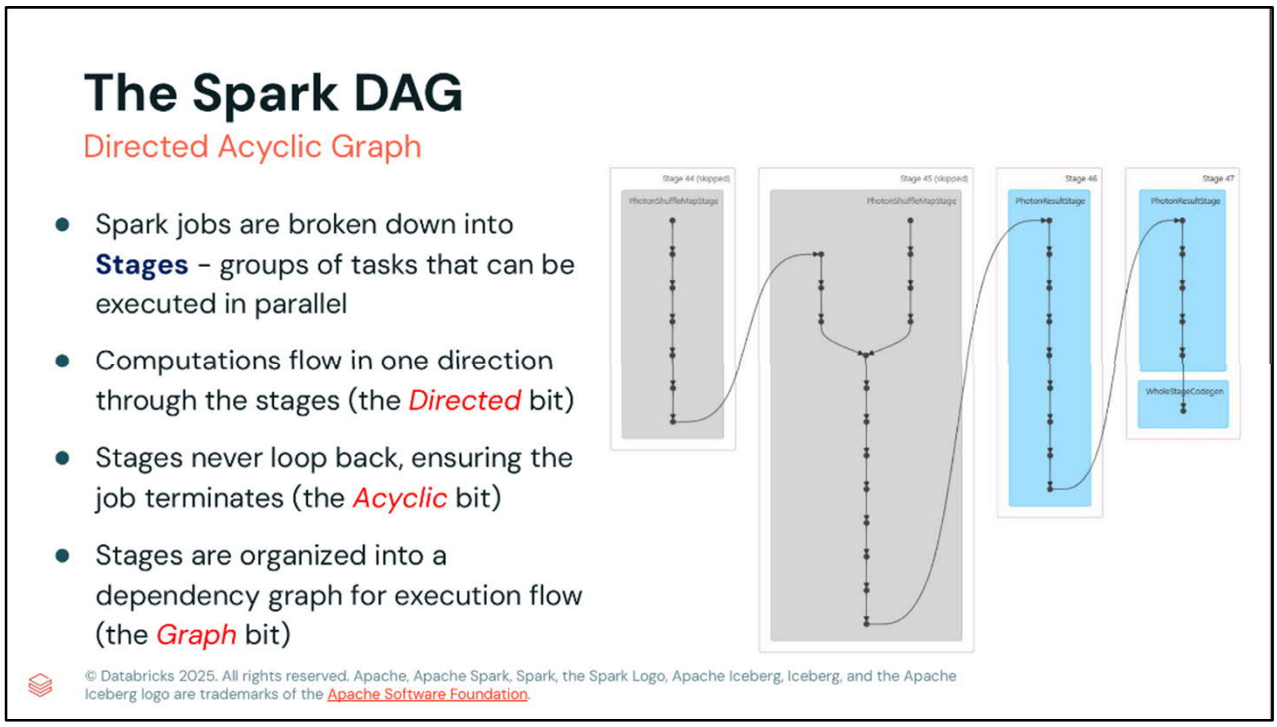
Manages the application lifecycle by creating the SparkSession for communication with the cluster. Analyzes jobs by constructing a Directed Acyclic Graph (DAG) for processing. Schedules tasks by distributing work to Executors. Monitors progress and handles failures to ensure fault tolerance and efficient resource use. Returns results to the client, ensuring end-to-end data processing.



Unified entry point introduced in Spark 2.0 as a consolidation of various contexts. Previously, users had to manage separate contexts like SparkContext, SQLContext, and HiveContext. Now, SparkSession encapsulates all functionalities. In Databricks, it is automatically created as spark.



Task execution and I/O handle data processing tasks and communicate results back to the driver. Executors run on worker nodes, with each node capable of hosting multiple executors. Resource management is configured through settings like spark.executor.cores and spark.executor.memory. Executors can store intermediate and final results in persistent storage. They interact with the driver by processing assigned tasks and reporting status.



Stages are groups of tasks that can be executed in parallel. Operations flow in a directed manner from one stage to the next, with no backward loops. This acyclic structure ensures computations complete successfully without infinite loops. A dependency graph is used to plan the execution flow for parallel processing.

1. What does "DAG" actually mean?

Directed: The process flows in one direction (from input data to final result).
Acyclic: There are no loops. You can't go back to a previous state once a transformation is done.
Graph: It's a mathematical structure of **Vertices** (the data/RDDs) and **Edges** (the operations like filter or map).

2. How the DAG is Built (The Lifecycle)

Spark doesn't run your code line-by-line. It uses **Lazy Evaluation**.

Transformations: When you write df.filter() or df.select(), Spark simply adds a "note"

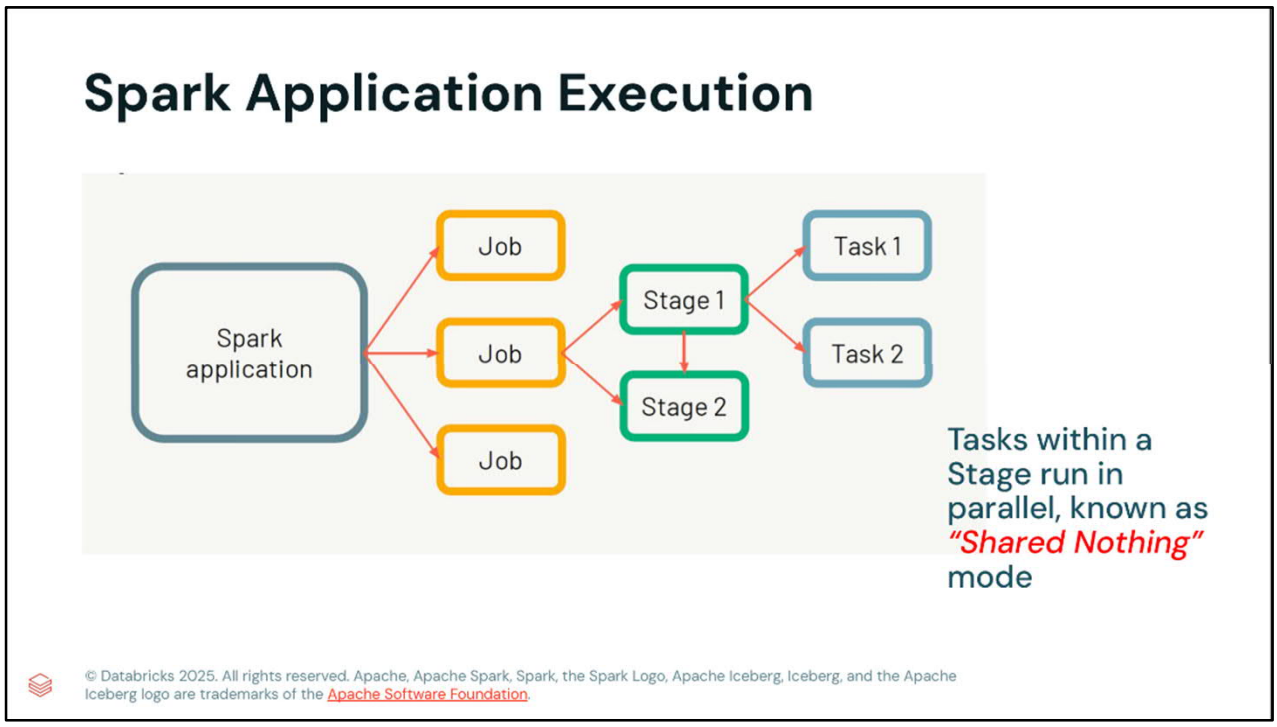
to the DAG. No data is moved yet.
Action: Only when you call an **Action** (like .show(), .count(), or .save()) does Spark look at the entire DAG.
Optimization: Spark's optimizer (Catalyst) looks for ways to combine steps (e.g., merging two filters into one) to save time.
Submission: The DAG is handed to the **DAG Scheduler**, which breaks it into **Stages** and **Tasks**.

3. The Hierarchy: Job → Stage → Task
To understand how a DAG is executed, you need to see how it's sliced:

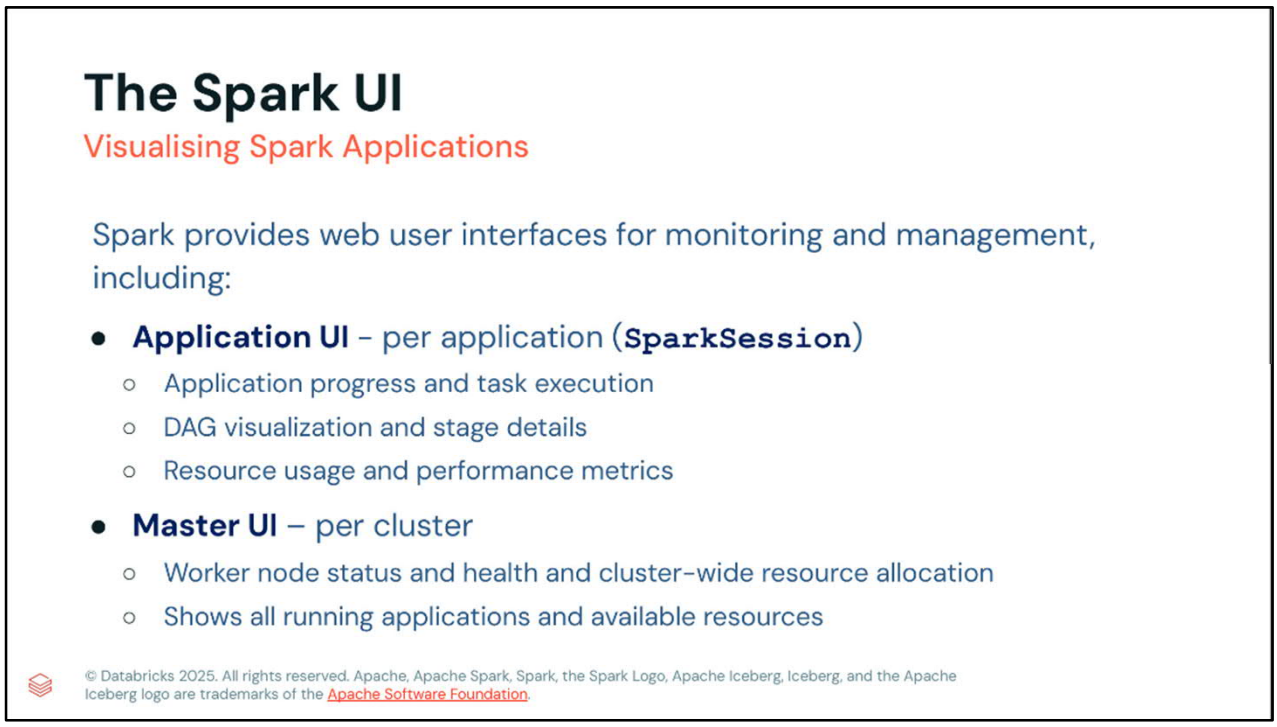
A. Jobs
Every time you call an **Action**, a new **Job** is created. If your script has three .show() calls, you will see three Jobs in the Spark UI.

B. Stages
Spark divides a Job into Stages based on **Shuffle Boundaries**.
Narrow Transformations: (e.g., map, filter) Operations that happen on a single partition of data stay in the **same stage**.
Wide Transformations: (e.g., groupBy, join) These require data to be "shuffled" across the network to different nodes. This **starts a new stage**.

C. Tasks
The smallest unit of work. Each stage is divided into **Tasks** based on the number of data partitions. If a stage has 10 partitions, Spark creates 10 tasks to run in parallel.

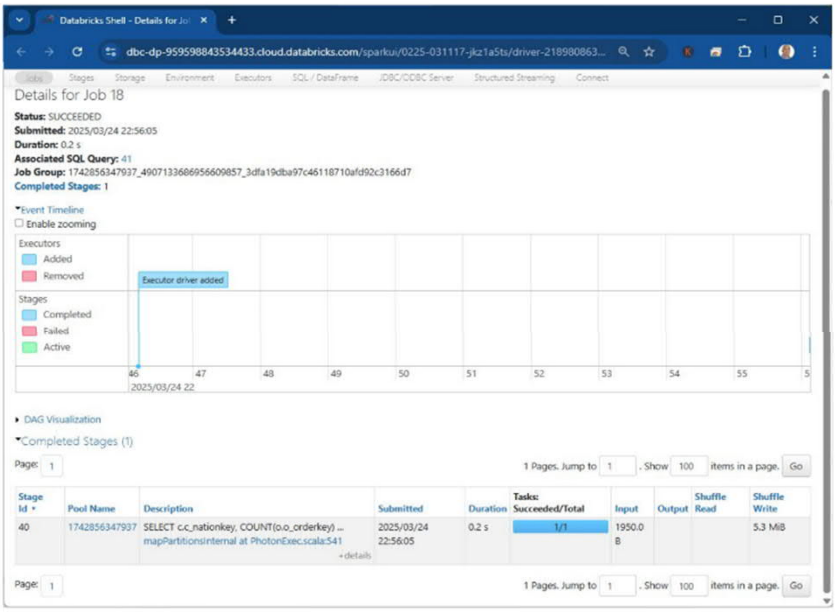


Jobs are high-level operations triggered by actions in Spark, such as collect() or save(). Stages divide jobs into smaller, independent units that can run in parallel. Tasks are the individual units of work executed by Executors. In parallel execution, tasks within a stage run independently using a "Shared Nothing" architecture.



Spark provides a user interface (UI) that allows you to monitor the Spark application, including metrics and visualizations of the DAG, resource usage, application progress, and more. There are two user interfaces: the Application UI, which is per application and monitored throughout the Spark session, and the Master Ul, which provides a clusterwide view of node health and resource allocation. Within the Databricks platform, these can be accessed during a demo. The Spark UI is particularly useful for troubleshooting and optimizing performance, identifying bottlenecks, debugging applications interactively, and more.

The Spark UI



The Application UI provides per-application monitoring through the SparkSession, offering details on progress, DAG visualization, resource usage, and more. The Master UI gives a cluster-wide view for monitoring multiple applications, showing the health status of nodes and overall resource allocation. Both interfaces are useful for interactive debugging, troubleshooting, and optimizing performance.